

INFORME DE CALIDAD Y PRUEBAS DE SOFTWARE

Suite de Pruebas de Integración — API REST ModaMágica
Backend Node.js / Express / PostgreSQL · Jest + Supertest

8	251	100%	2
Módulos analizados	Casos de prueba	Endpoints CRUD con prueba	Módulos con pruebas transaccionales

Proyecto: **ModaMágica** | Tipo de prueba: Integración (API HTTP con mocks de base de datos)
Fecha del informe: 29 de junio de 2026
Preparado por: Asistente de Calidad de Software (Claude)

1. Resumen Ejecutivo

La suite de pruebas automatizadas analizada para la API REST de **ModaMágica** está compuesta por **8 archivos de prueba de integración**, organizados en los módulos *Categorías*, *Descuentos*, *Pedidos*, *Colores*, *Imágenes*, *Productos*, *Tallas* y *Usuarios*. En conjunto suman **251 casos de prueba** ejecutados con **Jest** como runner y **Supertest** como cliente HTTP, simulando por completo la capa de acceso a datos (PostgreSQL) mediante *mocks* (`jest.mock('../../config/db')`).

El nivel de cobertura funcional es alto y consistente entre módulos: todos los endpoints implementan pruebas para el flujo exitoso (creación, lectura, actualización y eliminación), las validaciones de entrada, las reglas de negocio (duplicados, dependencias) y los códigos de error HTTP estándar (400, 404 y 500). En los módulos con lógica transaccional —**Pedidos** y **Productos**— se prueba además de forma explícita el comportamiento de **COMMIT/ROLLBACK** ante fallos intermedios dentro de una transacción, así como la liberación correcta del cliente de conexión.

No se identificaron pruebas frágiles evidentes ni dependencias de red reales: toda la suite es determinista gracias al uso sistemático de dobles de prueba (*mocks/stubs*). El principal espacio de mejora no es de cantidad sino de **tipo de prueba**: se recomienda complementar esta suite —de naturaleza unitaria/integración con *mocks*— con pruebas end-to-end contra una base de datos real, pruebas de seguridad/autorización y pruebas de concurrencia antes de un despliegue a producción. El detalle de estos hallazgos se desarrolla en las secciones 6 y 7 de este informe.

2. Alcance y Objetivo

Este informe documenta el análisis de calidad realizado sobre la suite de pruebas automatizadas de integración del backend del proyecto ModaMágica. Los objetivos específicos son:

- Cuantificar el número y el tipo de pruebas existentes por módulo funcional.
- Evaluar la cobertura funcional alcanzada: flujos CRUD, validaciones, manejo de errores y transacciones.
- Identificar fortalezas, vacíos y riesgos en la estrategia de pruebas vigente.
- Proponer recomendaciones concretas de mejora continua para el ciclo de calidad del proyecto.

El análisis se basa en la inspección estática de los 8 archivos de prueba ubicados en la estructura `tests/Categorías`, `tests/Descuentos`, `tests/Pedidos`, `tests/Productos` y `tests/Usuarios` del repositorio, según se observa en la captura de la estructura de carpetas proporcionada.

3. Metodología y Herramientas

Framework de pruebas. Jest, con `describe/test` anidados por endpoint y caso de uso.

Cliente HTTP simulado. Supertest, sobre instancias de Express montadas en memoria con los routers reales del proyecto (sin levantar un servidor en un puerto real).

Aislamiento de base de datos. `jest.mock('../../config/db')` sustituye por completo el pool de conexión (`pg.Pool`), permitiendo controlar de forma determinista las filas devueltas (`rows`) y forzar errores (`mockRejectedValueOnce`) en cada consulta SQL.

Aislamiento de dependencias externas. En el módulo de Imágenes se simulan además el sistema de archivos (fs) y el middleware de carga de archivos (multer / config/upload), evitando operaciones de E/S reales en disco.

Pruebas transaccionales. En Pedidos y Productos se utiliza un cliente simulado (`pool.connect()`) con métodos `query()` y `release()` encadenados, reproduciendo secuencias `BEGIN → ... → COMMIT/ROLLBACK` propias de transacciones SQL reales.

Aislamiento entre pruebas. `beforeEach(() => jest.clearAllMocks())` garantiza independencia total entre casos de prueba.

Reducción de ruido en consola. En Pedidos y Productos se silencia `console.error` durante la ejecución para mantener limpia la salida del runner sin perder la verificación de los códigos de error.

4. Resumen General de Resultados

Métrica	Valor
Archivos de prueba analizados	8
Módulos / entidades de negocio cubiertas	8 (Categorías, Descuentos, Pedidos, Colores, Imágenes, Productos, Tallas, Usuarios)
Total de casos de prueba (test)	251
Promedio de pruebas por módulo	≈ 31
Módulo con mayor número de pruebas	Usuarios (63 pruebas)
Módulo con menor número de pruebas	Tallas (13 pruebas)
Módulos con pruebas transaccionales (BEGIN/COMMIT/ROLLBACK)	Pedidos y Productos
Códigos de estado HTTP cubiertos de forma explícita	200, 201, 400, 404, 500

4.1 Distribución de pruebas por módulo

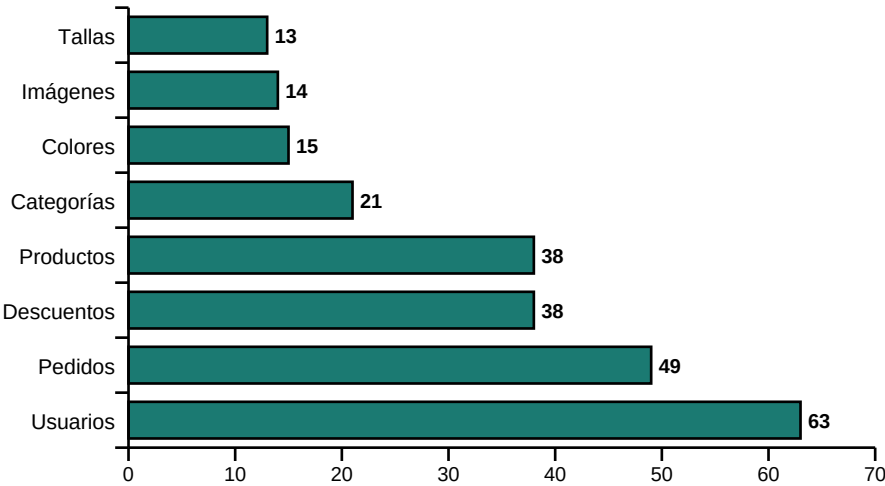


Figura 1. Número de casos de prueba por módulo (total = 251).

5. Distribución de Pruebas por Tipo

Cada uno de los 251 casos de prueba fue clasificado, a partir de su nombre descriptivo y su intención, en una de seis categorías funcionales. Esta clasificación permite entender en qué se concentra el esfuerzo de prueba actual:

Categoría	N° de pruebas	% del total
Validación de campos / reglas de negocio (400)	101	40.2%
Flujo exitoso (happy path)	68	27.1%
Manejo de errores de base de datos (500)	41	16.3%
Recurso no encontrado (404)	23	9.2%
Cobertura de ramas / valores por defecto	9	3.6%
Transaccional / Rollback	9	3.6%
TOTAL	251	100.0%

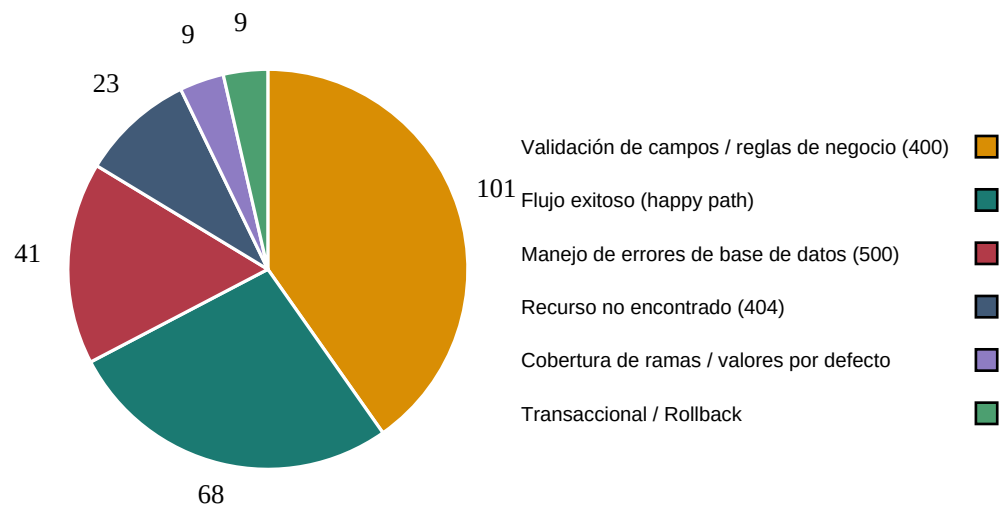


Figura 2. Distribución porcentual de los 251 casos de prueba por tipo.

La categoría dominante es **Validación de campos y reglas de negocio** (40.2% del total), seguida del **flujo exitoso** (27.1%). Esto indica que el equipo ha priorizado consistentemente la verificación de entradas inválidas y reglas de negocio por encima de la simple verificación de "camino feliz", una práctica deseable en una API que expone formularios de registro de clientes, productos y pedidos.

6. Detalle de Resultados por Módulo

La siguiente tabla desagrega los 251 casos de prueba por módulo y por tipo de prueba, permitiendo identificar rápidamente el perfil de cada suite (por ejemplo, Pedidos y Productos concentran la mayoría de las pruebas transaccionales, mientras que Usuarios concentra la mayor proporción de validaciones de campo).

Módulo	Archivo	Total	Validación (400)	Éxito	404	500	Trans./ Rama
Categorías	categorias.test.js	21	8	5	3	5	0
Descuentos	descuento.test.js	38	19	9	3	5	2
Pedidos	pedidos.test.js	49	14	18	4	10	3
Colores	colores.test.js	15	3	6	2	4	0
Imágenes	imagenes.test.js	14	2	6	2	4	0
Productos	productos.test.js	38	11	10	3	4	10
Tallas	tallas.test.js	13	3	4	2	4	0
Usuarios	usuarios.test.js	63	41	10	4	5	3
TOTAL		251	101	68	23	41	18

Nota: la columna "Trans./Rama" agrupa las pruebas de rollback transaccional (Pedidos, Productos) y las pruebas dedicadas a cobertura de ramas / valores por defecto (p. ej. parámetros opcionales, ternarios).

6.1 Hallazgos específicos por módulo

Categorías (21 pruebas). CRUD completo con control de duplicados por nombre y validación de formato (solo letras), longitud de nombre (≤ 100) y descripción (≤ 255). Cobertura equilibrada entre validación, éxito, 404 y 500.

Descuentos (38 pruebas). Incluye una suite específica para la función pura `validar()` invocada directamente (sin pasar por HTTP) para forzar el valor por defecto de `esEdicion`, lo cual maximiza la cobertura de ramas. Valida rangos numéricos (1–100), coherencia de fechas y manejo del código de error de unicidad de PostgreSQL (23505).

Pedidos (49 pruebas). El módulo más complejo en lógica de negocio: valida descuentos porcentuales y fijos, descuentos vencidos/no vigentes, control de stock insuficiente, todos los estados del ciclo de vida del pedido y republicación del cliente de conexión tras error. Es, junto con Productos, el único módulo con pruebas de ROLLBACK transaccional explícitas.

Colores (15 pruebas) e Imágenes (14 pruebas). Cobertura CRUD estándar; Imágenes añade pruebas de robustez frente al sistema de archivos (archivo físico inexistente, nombre de archivo nulo) gracias al mock de `fs` y `multer`.

Productos (38 pruebas). Maneja variantes de color/talla con varias combinaciones de payload (con y sin tallas, con y sin `stock_actual`). Incluye un bloque adicional de "cobertura extra" dedicado a verificar que `client.release()` se invoque siempre, incluso en caminos de error.

Tallas (13 pruebas). El módulo más simple del dominio; cobertura CRUD básica y validación de nombre obligatorio y duplicado.

Usuarios (63 pruebas). El módulo con mayor densidad de pruebas de validación (41), cubriendo expresiones regulares de nombre/apellido, formato y longitud de correo, formato de teléfono, política de contraseña (longitud, mayúscula, minúscula, número), validez de rol y cálculo de edad mínima (18 años), incluyendo casos límite de fecha de cumpleaños (mismo mes, antes/después del día actual).

7. Fortalezas Identificadas

- Cobertura sistemática del patrón CRUD completo (GET all, GET by id, POST, PUT, DELETE) en los 8 módulos analizados.
- Pruebas explícitas para todos los códigos de estado relevantes: 200/201 (éxito), 400 (validación/negocio), 404 (no encontrado) y 500 (error interno).
- Validaciones de campo exhaustivas, en especial en Usuarios y Descuentos: formato, longitud mínima/máxima, obligatoriedad, expresiones regulares (correo, teléfono, nombre, código de color hexadecimal) y reglas de fecha/edad.
- Estrategia deliberada de cobertura de ramas: existen pruebas dedicadas a ejecutar rutas de código por defecto en parámetros opcionales (p. ej. `esEdicion`, `id_usuario_cambio` `|| 1`, `estado` `|| 'activo'`, `stock_actual` `|| 0`), lo que evidencia una mentalidad orientada a cobertura de código y no solo de comportamiento.
- En los módulos transaccionales (Pedidos, Productos) se prueba explícitamente el ROLLBACK ante fallos de consulta intermedios y la liberación del cliente de conexión en todos los caminos, mitigando fugas de conexiones a la base de datos.
- Casos de negocio complejos cubiertos en Pedidos: descuentos porcentuales frente a fijos, descuentos vencidos o no vigentes al momento de crear el pedido, control de stock insuficiente y los cinco estados válidos del ciclo de vida del pedido.
- Pruebas de robustez frente a dependencias externas en Imágenes: eliminación de imagen cuando el archivo físico no existe en disco o cuando el nombre de archivo es nulo.
- Prueba unitaria directa de la función `validar()` de Descuentos (importada del controlador, sin pasar por HTTP), una buena práctica para cubrir ramas difíciles de alcanzar únicamente a través de la API.

8. Riesgos y Áreas de Mejora

- **Ausencia de pruebas end-to-end:** toda la suite analizada simula la base de datos; no hay evidencia de pruebas contra una instancia real (ni siquiera en un contenedor de pruebas), por lo que errores de esquema SQL (tipos de columna, restricciones, nombres) no serán detectados por esta suite.
- **Sin pruebas de autenticación/autorización:** ninguno de los 8 archivos ejercita control de acceso (roles, JWT, sesiones), pese a que el módulo Usuarios gestiona contraseñas e `id_rol`.
- **Sin pruebas de concurrencia:** no se evidencian casos de dos solicitudes simultáneas descontando el mismo stock de inventario, un riesgo relevante en el flujo de creación de pedidos.
- **Cobertura desigual entre módulos:** Tallas e Imágenes presentan significativamente menos pruebas (13–14) que Usuarios (63); esto puede ser razonable según la complejidad de negocio de cada entidad, pero conviene confirmarlo con un reporte de cobertura de líneas/ramas real (`jest --coverage`) en lugar de solo el número de pruebas.
- **Acoplamiento a la implementación:** la dependencia de `mockResolvedValueOnce()` encadenado en orden estricto significa que, si cambia el orden interno de las consultas SQL del controlador, las pruebas pueden romperse sin que exista un defecto funcional real.
- **Sin pruebas de payloads malformados:** no se evidencian casos con tipos de datos incorrectos en el cuerpo JSON (por ejemplo, arreglos donde se espera un número, u objetos anidados inválidos), ni de límites de

tamaño de payload.

9. Recomendaciones

1. Ejecutar la suite con `jest --coverage` e incorporar un umbral mínimo (por ejemplo, 85% de líneas y 80% de ramas) como puerta de calidad (quality gate) en el pipeline de integración continua.
2. Incorporar un conjunto reducido de pruebas end-to-end con una base de datos de pruebas real (PostgreSQL en un contenedor Docker efímero) que ejecute las migraciones reales antes de cada corrida, validando el contrato SQL completo.
3. Añadir pruebas de seguridad: verificación de control de acceso por rol, expiración de sesión/token y comprobación de que `bcrypt.hash` se invoque siempre antes de persistir una contraseña.
4. Sumar pruebas de concurrencia para el descuento de inventario en Pedidos, simulando una carrera entre dos solicitudes simultáneas sobre el mismo producto/talla.
5. Documentar en el repositorio (por ejemplo, en un README de la carpeta `tests/`) la convención de comentarios “Línea N — ...” usada para mapear pruebas a líneas de código, e idealmente enlazarla con el reporte HTML de cobertura generado por Istanbul.
6. Estandarizar en todos los módulos la supresión controlada de `console.error` durante las pruebas (ya aplicada en Pedidos y Productos), para mantener una salida de test runner limpia sin perder la verificación de errores.
7. Evaluar la migración progresiva de las validaciones de campo repetidas (correo, teléfono, fechas, longitud de texto) hacia un esquema de validación declarativo (por ejemplo, Joi o Zod) con sus propias pruebas unitarias, reduciendo la duplicación de lógica entre Categorías, Descuentos, Productos y Usuarios.

10. Conclusión

La suite de pruebas evaluada demuestra un nivel de madurez alto para pruebas de integración basadas en dobles de prueba (mocks): cobertura CRUD completa en los 8 módulos del dominio, manejo exhaustivo de validaciones y códigos de error HTTP, y atención específica tanto a la cobertura de ramas de código como a la integridad transaccional en los módulos críticos de negocio (Pedidos y Productos).

El vacío principal identificado no es de *cantidad* de pruebas sino de *tipo*: se recomienda complementar —no reemplazar— esta suite con pruebas end-to-end contra una base de datos real, pruebas de seguridad/autorización y pruebas de concurrencia, de modo que la estrategia de calidad cubra de manera integral tanto la lógica de los controladores como el comportamiento del sistema en condiciones reales antes de un despliegue a producción.